Feature Flags

deploy is code, release is config

Watch the 4-min video

In one line: feature flags separate deploy from release: code ships switched off and is turned on, targeted, experimented with or killed by a configuration change in seconds; the price is extra code paths and flag debt, so every flag needs an owner, an expiry date and audited changes.

1 The LLM provider is down

2 p.m., your LLM provider has an outage. The AI summary feature times out, and slow requests pile up across the whole app, exhausting threads and connections. You could hot-fix and redeploy: half an hour. Or flip one switch and turn the feature off in seconds.

2 Decoupling deploy from release

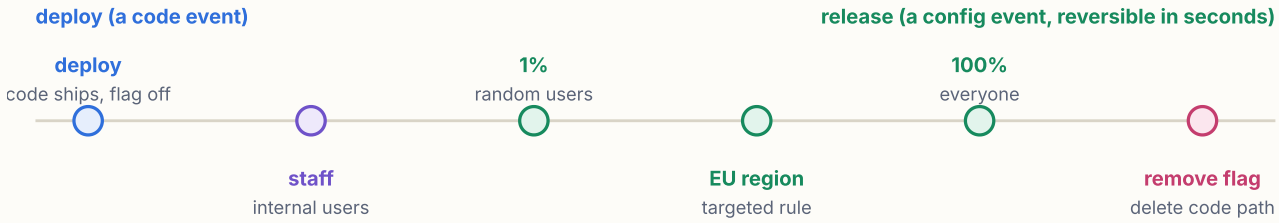


Figure 1. A feature's life behind a flag: deployed dark, released step by step by changing rules, then the flag is removed.

Code ships to production hidden behind a flag. Releasing becomes a configuration change, not a deployment, and rolling back is the same change in reverse. Teams can merge small changes to main every day (trunk-based development), even for unfinished features (lesson 72).

3 Four kinds of flag

Type	Purpose	Lifetime	Example
Release	hide unfinished or unreleased work	days to weeks	new checkout page
Ops / kill switch	turn off expensive or risky features under load or outage	long-lived	`ai_summary_enabled`
Experiment	split users for A/B tests	until the result is in	two ranking models
Permission	features per plan or customer	long-lived	SSO for enterprise plan

This classification follows Pete Hodgson's "Feature Toggles" article on martinfowler.com. Knowing the type tells you who owns it and when to delete it.

4 Targeting and evaluation

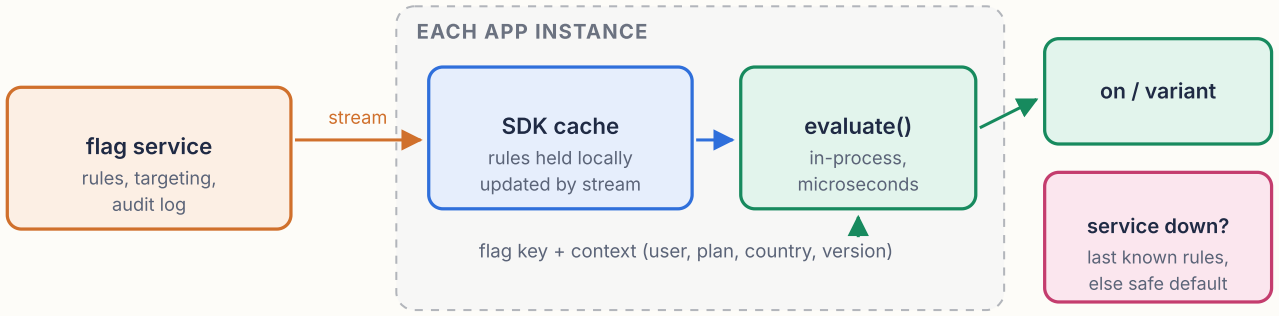


Figure 2. SDKs keep the rules locally and evaluate in-process, so a flag check costs microseconds and survives a flag-service outage.

Targeting decides who gets what: staff first, then 1% of users, then a region, then everyone; any attribute you pass (plan, country, app version) can be used. Percentage rollouts hash a stable key (user ID + flag key) into buckets, so a user stays in the same group. Evaluation must be fast and reliable: SDKs cache rules and update them in the background. If the flag service is unreachable, they use the last known rules, and on a cold start the **default you pass in code**, so choose a safe one.

5 Risks and discipline

- **Combinations:** n boolean flags make 2^n possible states; most were never tested together.
- **Flag debt:** old flags pile up until nobody knows which are safe to delete.
- **Flag changes are production changes:** a mistaken toggle causes outages just like a bad deploy.

So: every release flag gets an **owner and an expiry date** and is removed after full rollout; some teams fail the build when a flag is past expiry. Flag changes are **audited and reviewed** like code. Use a flag service (LaunchDarkly, Unleash, Azure App Configuration) behind the vendor-neutral **OpenFeature** API, rather than ad-hoc `if` statements on environment variables that need a redeploy to change.

```
client = openfeature.api.get_client()
def summary(doc):
    if not client.get_boolean_value("ai_summary_enabled", False):   # safe default: off
        return plain_text(doc)                                       # fallback
    return llm.summarize(doc, timeout=3)
```

Figure 3. The kill switch from the video, written against the OpenFeature API so the flag provider can be swapped.

WORKED EXAMPLE: A KILL SWITCH FOR AN LLM FEATURE

Every call to the summary feature first checks `ai_summary_enabled`. When the provider starts failing, the on-call engineer flips it off (it is in the runbook, lesson 67). Within seconds, every instance's SDK receives the change, stops calling the model and returns the plain text without a summary. Timeouts stop, threads free up, the rest of the app recovers. When the provider is back, the flag goes on again, with no deploy in either direction.

6 Key terms

Feature flag: runtime switch that changes behaviour without a deploy

Kill switch: ops flag that disables a feature fast

Targeting rule: who gets which variation, by attributes

Local evaluation: SDK evaluates cached rules in-process

Flag debt: stale flags and dead code paths left behind

OpenFeature: vendor-neutral feature-flag API (CNCF)

7 Interview questions

Q1. What are the risks of feature flags? BASIC

Untested flag combinations, growing flag debt, more complex code, and flag changes that can cause outages like bad deploys; so give flags owners, expiry dates and audited changes.

Q2. Why use feature flags at all? BASIC

They separate deploy from release, enable gradual rollouts and experiments, and give you instant kill switches.

Q3. A third-party API your feature depends on is down and dragging the app. What do you do? SCENARIO

Flip the feature's kill switch to its fallback, which stops the slow calls at once; combine with timeouts and circuit breakers (lessons 4, 6) so it degrades automatically next time.

Q4. The flag service goes down. What happens to your app? SCENARIO

Nothing visible if designed well: SDKs keep evaluating the last known rules locally; only new instances fall back to code defaults.

Q5. Feature flag or canary for a new version of a service? TRICKY

Canary for the whole binary and infrastructure-level risk; flags for specific features inside it, with per-user targeting and instant off. Many teams use both.

Q6. Why must percentage rollouts hash a stable key instead of random()? TRICKY

Random per request would flip a user between variations on every call; hashing user ID + flag key keeps each user consistently in or out.

Next: Phase 6 · Advanced and Specialized Systems backendroadmap.com